

AWS Auto Scaling - Service Overview

Introduction

AWS Auto Scaling monitors your applications and automatically adjusts capacity to maintain steady, predictable performance at the lowest possible cost. An Auto Scaling Group (ASG) is a collection of EC2 instances treated as a logical grouping for the purposes of automatic scaling and management.

Launch Templates

An Auto Scaling Group uses a Launch Template (the modern replacement for the older Launch Configuration) to determine the configuration of instances it creates, including the AMI, instance type, key pair, security groups, and user data script used to bootstrap the instance on first boot.

Scaling Policies

- Target Tracking Scaling: the simplest and most commonly recommended policy, where you specify a target value for a metric (e.g. "keep average CPU utilization at 50%"), and Auto Scaling automatically adjusts capacity to maintain that target.
- Step Scaling: adjusts capacity by varying amounts based on the magnitude of the alarm breach, allowing more aggressive scaling response for larger deviations from the threshold.
- Scheduled Scaling: adjusts the minimum, maximum, and desired capacity based on a predictable schedule, useful for known traffic patterns such as business-hours-only workloads.
- Predictive Scaling: uses machine learning to forecast future traffic based on historical patterns and proactively scales capacity ahead of anticipated demand.

Minimum, Maximum, and Desired Capacity

Every Auto Scaling Group has three capacity settings: minimum (the floor, below which the group will never scale down), maximum (the ceiling, above which it will never scale up regardless of demand), and desired (the target the group currently attempts to maintain, adjusted automatically by scaling policies within the min/max bounds).

Health Checks

Auto Scaling Groups continuously monitor the health of instances within the group, using either EC2 status checks or, when attached to a load balancer, the load balancer's health checks. An unhealthy instance is automatically terminated and replaced with a new one, helping maintain the desired capacity of healthy instances at all times.

Multi-AZ Distribution

When configured to span multiple Availability Zones, an Auto Scaling Group automatically attempts to distribute instances evenly across the specified AZs, improving fault tolerance so that the loss of a single AZ does not take down the entire fleet.

Lifecycle Hooks

Lifecycle hooks allow you to perform custom actions (such as downloading configuration files or draining connections) as instances launch or terminate, by pausing the instance in a wait state before it transitions into service or is fully terminated.

Integration with ECS

ECS Service Auto Scaling operates conceptually similarly to EC2 Auto Scaling Groups, but adjusts the desired count of running tasks within an ECS service, rather than the number of EC2 instances, and can be layered on top of an underlying EC2-based ECS cluster's own capacity provider auto scaling.

Cooldown Periods

After a scaling activity occurs, Auto Scaling applies a cooldown period (default 300 seconds for simple scaling policies) during which further scaling activities triggered by the same alarm are suppressed. This prevents the group from launching or terminating instances too aggressively in response to metric fluctuations that occur while previously launched instances are still starting up and have not yet begun handling traffic. Target tracking policies manage this dynamically and generally require less manual cooldown tuning than simple or step scaling policies.

Warm Pools

A warm pool maintains a group of pre-initialized EC2 instances outside of the Auto Scaling Group's active capacity, in a stopped or pre-booted state, ready to be quickly moved into service when scale-out is needed. This significantly reduces the time needed to bring new capacity online compared to launching and fully bootstrapping a brand new instance from scratch, which is particularly valuable for applications with lengthy startup or configuration routines.

Common Use Cases

- Automatically handling variable web traffic without manual capacity planning
- Ensuring a minimum fleet size is always running for high availability
- Cost optimization by scaling down non-critical environments outside business hours
- Combining with Spot Instances to reduce compute costs for fault-tolerant workloads

Best Practices

- Prefer target tracking scaling policies for most workloads, since they self-tune more reliably than manually configured step policies.
- Set a sensible minimum capacity greater than zero for production workloads to avoid a cold-start gap during a sudden traffic spike.
- Span at least two Availability Zones for any production Auto Scaling Group.
- Use lifecycle hooks to ensure in-flight requests are drained gracefully before an instance is terminated during scale-in.